

Post-Trigger Artillery Launch Classifier

[0:00–0:40]

Good morning. // My name is Martin Maxa, and I'm from the Czech Technical University in Prague, the Faculty of Electrical Engineering. // This is joint work with my colleagues Jakub Svatoš and Jakub Zelinka. //

Today I'll show you a small neural network that recognizes the sound of an artillery launch. // It runs on a cheap microcontroller, and it keeps working in noise. //

(// = pause. [CLICK] = advance the slide.)

Post-Trigger Artillery Launch Classifier

└ Motivation

└ Motivation – decide on the edge, in noise

- Passive acoustic sensing supports artillery early warning: it is **cheap**, needs **little power** and **emits no signal** – active systems are expensive, power-hungry and easier to locate.

■ The decision has to be on the edge

- Cheap nodes mean many nodes, spread over a large area.
- Streaming raw audio from every node costs power and radio bandwidth – and a transmitting node is easier to find.
- So each node decides by itself and sends an alert, not the audio.

⚡ The signal arrives weak

- Artillery fires from far away – in our recordings the guns were ~5-10 km from the station.
- The launch reaches the sensor attenuated, on top of wind, traffic and other background.
- So the classifier has to work at **low SNR**.

[0:40–1:50]

Let me start with the motivation. //

Recent conflicts have shown that we need cheap systems that can warn about artillery fire. //
Acoustic sensing is a good candidate. // It is passive, it needs little power, and it doesn't send out any signal. //

But if the sensor is cheap, we want many of them. // And then we cannot stream raw audio from every node to a server. // It costs power and bandwidth, and a node that transmits all the time is easy to find. // So the decision has to be made on the edge – on the node itself. // The node sends only an alert. //

The second problem is the signal. // Artillery usually fires from far away – in our recordings, five to ten kilometres. // So the launch arrives weak, and the signal-to-noise ratio is low. // The classifier has to work in noise. //

Post-Trigger Artillery Launch Classifier

└ Motivation

└ Research questions



low SNR

Q1 Where to add the training noise?

Can cheap **MFCC-domain** augmentation replace **waveform-domain** noise augmentation for a compact post-trigger classifier?



on the edge

Q2 Does it survive the chip?

Does the resulting robustness survive **int8 quantization** and deployment on an **MCU** (ESP32-S3)?

[1:50–2:30]

These two problems give us two questions. //

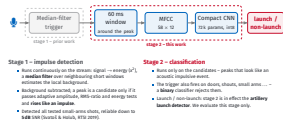
First, low SNR. // To make the classifier robust, we add noise during training. // Can we add it cheaply, to the MFCC features? // Or do we have to add it to the audio signal itself? //

Second, the edge. // Does this robustness survive when we quantize the model to int8 and run it on a microcontroller? //

Post-Trigger Artillery Launch Classifier

└ Pipeline

└ Two-stage detection pipeline



[2:30-3:50]

Our system has two stages. //

The first stage is our earlier work. // It runs all the time on the audio stream. // It turns the signal into energy, and a median filter over the neighbouring windows estimates the background noise. // The background is subtracted, and a peak becomes a candidate only if it is clearly above the background and rises like an impulse. //

This trigger detected all the gunshots we tested, and it works down to five decibels. // But it fires on every impulsive sound – a door, a shout, a rifle shot. //

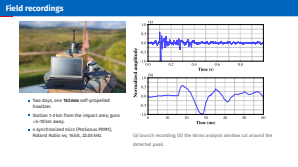
That's why we need the second stage. // Only the candidates go there. // We cut a sixty millisecond window around the peak, compute MFCC features, and a small CNN decides: artillery launch, or not. // So the binary classifier is, in effect, the artillery launch detector. //

In this talk we evaluate only the second stage. //

Post-Trigger Artillery Launch Classifier

└ Data

└ Field recordings

**[3:50-4:50]**

The data come from real artillery measurements. //

This is our recording station. // Four synchronized microphones on one mount, an audio interface and a laptop. //

We recorded for two days, with one type of gun – a 152 millimetre self-propelled howitzer. // The station was one to two kilometres from the impact area, and the guns fired from about five to ten kilometres away. //

On the right is one launch. // Below it, you can see the sixty millisecond window that the classifier actually sees. // Thirty percent of it is before the peak, and seventy percent after it. //

Post-Trigger Artillery Launch Classifier

└ Data

└ Labelled corpus and event-level split

Subent	Events	Samples		
		Train	Val.	Test
Artillery launch	50	128	8	90
Impulsive noise	704	407	109	140
Small-arms shots	85	40	18	18
Total	841	624	125	168

- Non-launch: false-trigger candidates – door impacts, shouts, bells, **small-arms gunshots** as hard negatives.
- Split by event: the four channels of one shot never cross partitions.
- Training uses all 4 channels (launch share 20 %, class-weighted loss); validation and test use one.

[4:50–5:40]

This is the whole labelled corpus – eight hundred and forty-one events. //

Fifty of them are artillery launches. // The rest are things that can also fire the trigger – doors, shouts, bells, and small-arms gunshots. // The gunshots are the hard cases, because they are impulsive too. //

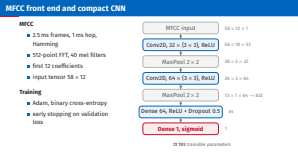
We split the data by event, not by recording. // So the four channels of one shot are never in training and test at the same time. //

In training we use all four channels, and a class-weighted loss for the imbalance. // The test set has ten launches and one hundred and fifty-eight other events. //

Post-Trigger Artillery Launch Classifier

Method

MFCC front end and compact CNN



[5:40–6:40]

Now the method. //

From each window we compute MFCCs. // We use very short frames – two and a half milliseconds, with a one millisecond step – because the event is short and changes fast. // We keep twelve coefficients, so each event becomes a small matrix, fifty-eight by twelve. //

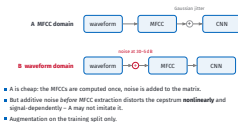
The classifier is a small CNN. // Two convolution blocks, each with max pooling, then one dense layer with dropout, and a sigmoid output. //

In total it has about seventy-two thousand parameters. // That is small enough for a microcontroller. //

Post-Trigger Artillery Launch Classifier

└ Method

└ Two ways to train for noise

**[6:40–7:40]**

To make the network robust, we add noise during training. // And there are two places where we can add it. //

Option A: we compute the MFCCs once, and then add small random noise directly to the feature matrix. // This is cheap and simple. //

Option B: we add noise to the audio signal first, and only then compute the MFCCs. // This costs more, because we have to recompute the features every time. //

The question is whether A is good enough. // Noise in the audio does not stay simple after the MFCC – it changes the features in a nonlinear way. // So A may not look like real noise at all. //

Post-Trigger Artillery Launch Classifier

└ Method

└ Evaluation protocol

- Held-out test partition, used only for the final numbers.
- Noise added to the **waveform before MFCC extraction**, per event:

$$\sigma_n^2 = P_s \cdot 10^{-SSB/10}$$

at **30, 20, 10, 5 dB** + plus 0 and -5 dB as a stress test, beyond the trigger's 5 dB envelope.

- Peak detection and windowing re-run on the noisy signal.
- **5 training seeds** × **5 noise seeds** = 25 runs per cell, mean ± std; threshold 0.5.
- Three regimes: **float32** and **int8** on the PC, **int8** on the **ESP32-S3**.
- Main metric: Matthews correlation coefficient (MCC).

[7:40–8:30]

How do we test it? //

We take the held-out test set and add noise to the audio – again before the MFCC, because that's what happens in the real sensor. // The noise level is set per event, from the power of the event itself. //

We test thirty, twenty, ten and five decibels. // And then zero and minus five, which is already beyond what the trigger can handle. //

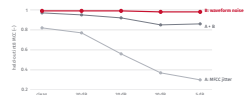
Every number is an average over twenty-five runs – five training seeds and five noise seeds. // And we report mainly the MCC, because the classes are imbalanced. //

Post-Trigger Artillery Launch Classifier

Results

Result 1 – where to add the noise

Result 1 – where to add the noise



Same network, three augmentation strategies, mean over 25 runs. B: 1.00 ± 0.01, A + B: 0.88 ± 0.04, A: 0.35 ± 0.04.

[8:30–9:40]

This is the first result. // Same network, trained three times – only the augmentation is different. //

The light grey line is option A, noise on the features. // It is fine on clean data, but it falls apart in noise. // At five decibels the MCC is only zero point three. //

The red line is option B, noise on the waveform. // It stays almost flat – zero point nine eight even at five decibels. //

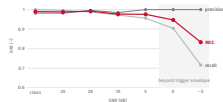
And the surprise is the middle line. // When we combine both, it is worse than B alone. // So feature noise is not just insufficient – here it actually hurts. //

Post-Trigger Artillery Launch Classifier

Results

Result 2 – held-out robustness of the deployed model

Result 2 – held-out robustness of the deployed model



Precision stays 100% from 5 dB down – in heavy noise the model misses launches, it does not raise false alarms, focused vs. data within +0.02 MCC, no consistent direction.

[9:40–10:50]

Now the deployed model in more detail. //

In the trained range, from clean down to five decibels, the MCC stays between zero point nine eight and zero point nine nine. //

In the grey area, beyond what the trigger can handle, it goes down slowly. // Zero point nine five at zero decibels, and zero point eight three at minus five. //

But look at how it fails. // Precision stays at one. // Only recall drops. // So in very strong noise the model misses some launches – but it does not give false alarms. // And no small-arms gunshot was ever classified as artillery. //

Quantization to int8 made no real difference. //

Post-Trigger Artillery Launch Classifier

Results

Result 3 – on the ESP32-S3

[10:50–11:50]

Finally, the microcontroller. //

We sent all test inputs to the ESP32-S3, at all noise levels – that's one thousand one hundred and seventy-six inferences. // One thousand one hundred and seventy-one gave exactly the same result as the PC. // The other five had a score of exactly zero point five, right on the decision boundary, where a one-bit rounding difference can flip the answer. //

One inference takes thirty-two milliseconds. // Please note, that's only the network – the MFCCs were computed on the host. //

The model has about eighty kilobytes, and the whole firmware needs less than one hundred and fifty kilobytes of RAM. //

Result 3 – on the ESP32-S3

Same answers as the PC

137 of 176 inferences bit-exact with PC int8.
The 5 others scores at exactly 0.5 – a legal one-bit rounding difference between TFLite and TFLite Micro kernels.

Latency

32.0 ms per MFCC segment (31.99–32.09 ms), independent of noise.

Classifier inference only – MFCC computed on the host.

TensorFlow Lite Micro, int8

Model size	87,400 B
tensor_arena	80 KiB
Flash - code	201.5 KiB
Flash - data	144.0 KiB
RAM	143.8 KiB

All 8 operations native to TFLite Micro, compute-heavy ones via ESP-48B kernels.

Post-Trigger Artillery Launch Classifier

└ Discussion

└ Limitations

- **Stage 2 only** – no continuous-stream false-trigger rate; results assume the trigger found the peak.
- **Small positive class** – 50 launch events, one gun type (152 mm); other calibres untested.
- **Synthetic Gaussian noise** – closer to reality than feature jitter, but no field noise, propagation or device shift.
- **One event-level split**; no session-independent evaluation.
- **MFCC on the host** – on-device MFCC, end-to-end latency and energy per decision are future work.

[11:50–12:20]

Of course, there are limitations. //

We evaluate only the classifier, not the full stream. // We have only fifty launches, and one type of gun. // And the noise is synthetic. //

So these are the next steps for us. //

Post-Trigger Artillery Launch Classifier

└ Conclusion

└ Conclusion

- A compact MFCC-CNN (72k parameters) works as the post-trigger stage of an embedded artillery launch detector.
- **Add the noise to the waveform, not to the features:** MFCC-domain jitter collapses at 5 dB (MCC 0.30) and even hurts when combined.
- Waveform-trained model: **MCC 0.99 at 30 dB, 0.98 at 5 dB**; precision 1.0 under heavy noise.
- int8 is performance-neutral; ESP32-S3 matches the PC, 32 ms per inference.

Thank you – questions?

manamart@fel.cvut.cz

[12:20–13:00]

To sum up. //

A small CNN with seventy-two thousand parameters works well as the second stage of an artillery launch detector. //

If you want it robust to noise, add the noise to the audio, not to the features. // Feature noise fails, and combined with waveform noise it even makes things worse. //

Our model keeps an MCC of zero point nine eight at five decibels, and it runs on an ESP32-S3 in thirty-two milliseconds. //

Thank you for your attention. // I'm happy to take your questions. //